PYTHON ACTIVITY-BASED LEARNING REPORT

Name: Thanuj Lanke	**Roll No:** 26000175
Branch: CSE	**Course:** Python Programming

Case 1: Progressive Electricity Billing

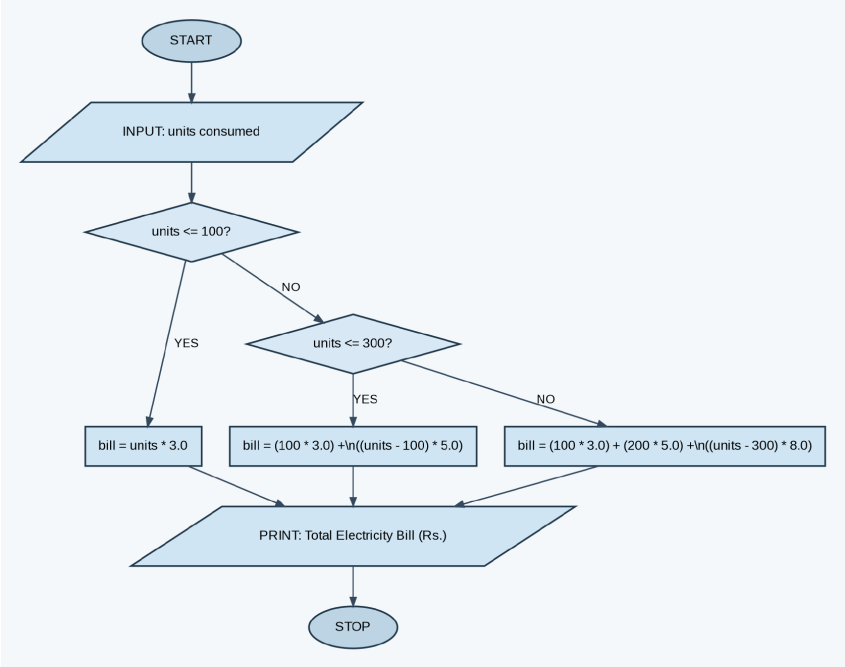
Problem Statement

Write a Python program to calculate the progressive electricity bill based on units consumed using tiered pricing rules (Rs. 3.0 per unit for the first 100 units, Rs. 5.0 for the next 200 units, and Rs. 8.0 for every unit above 300).

Algorithm

- Step 1: Start.
- Step 2: Read units consumed from user.
- Step 3: If units ≤ 100, bill = units × 3.0.
- Step 4: Else if units ≤ 300, bill = (100 × 3.0) + ((units - 100) × 5.0).
- Step 5: Else bill = (100 × 3.0) + (200 × 5.0) + ((units - 300) × 8.0).
- Step 6: Print total bill. Stop.

Flowchart



Source Code & Output

```
units = int(input("Enter units consumed: "))
if units <= 100:
    bill = units * 3.0
elif units <= 300:
    bill = (100 * 3.0) + ((units - 100) * 5.0)
else:
    bill = (100 * 3.0) + (200 * 5.0) + ((units - 300) * 8.0)
print("Total Electricity Bill: Rs.", bill)
```

```
Enter units consumed: 350 | Total Electricity Bill: Rs. 1700.0
```

Case 2: Employee Payroll Analysis

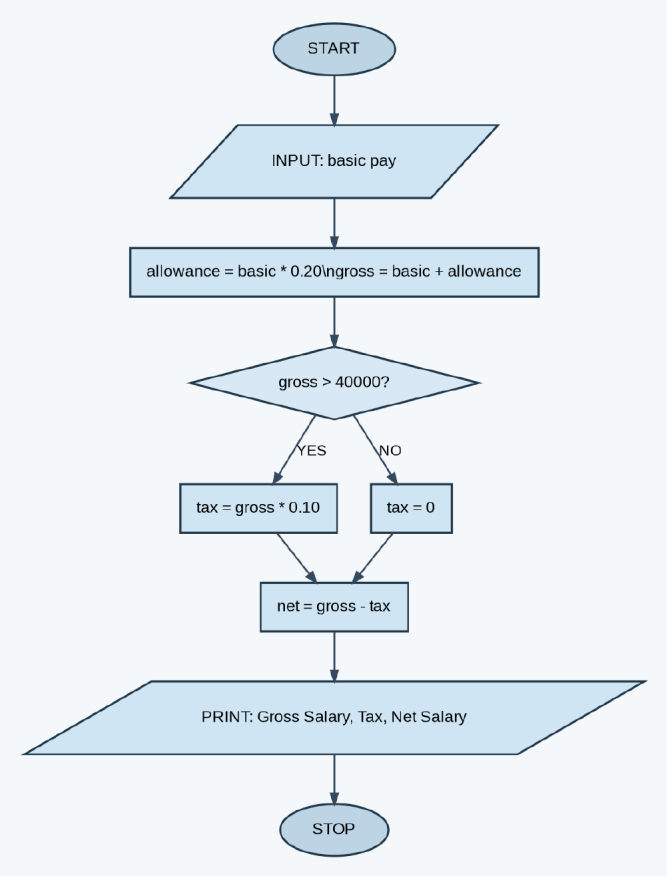
Problem Statement

Calculate gross salary, tax deductions based on income slab, and net salary for an employee. A 20% allowance is added to the basic pay, and 10% tax is deducted if the gross salary exceeds Rs. 40,000.

Algorithm

- Step 1: Start.
- Step 2: Read basic pay from user.
- Step 3: Calculate allowance = basic × 0.20.
- Step 4: Calculate gross = basic + allowance.
- Step 5: If gross > 40000, tax = gross × 0.10; otherwise tax = 0.
- Step 6: Calculate net = gross - tax.
- Step 7: Print gross salary, tax and net salary. Stop.

Flowchart



Source Code & Output

```
basic = float(input("Enter basic pay: "))
allowance = basic * 0.20
gross = basic + allowance
tax = gross * 0.10 if gross > 40000 else 0
net = gross - tax
print("Gross Salary:", gross, "Tax:", tax, "Net:", net)
```

```
Enter basic pay: 10000 | Gross Salary: 12000.0 | Tax: 0 | Net: 12000.0
```

Case 3: Railway Fare Calculator

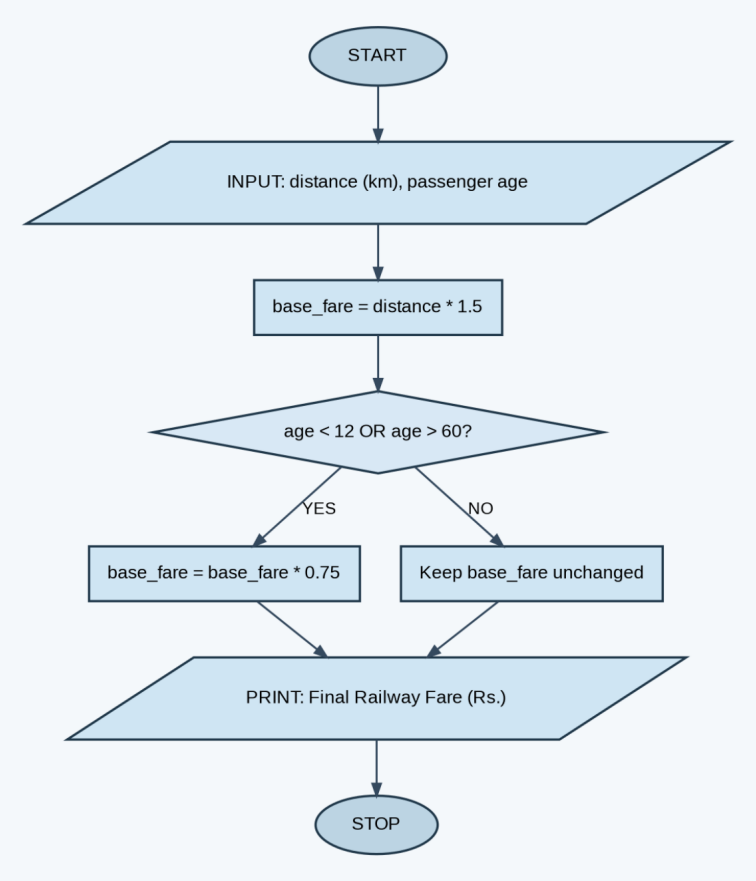
Problem Statement

Write a Python program to calculate the railway fare for a passenger from the distance travelled and the passenger's age. The base fare is Rs. 1.5 per km, and children below 12 years and senior citizens above 60 years get a 25% concession.

Algorithm

- Step 1: Start.
- Step 2: Read distance (km) and passenger age from user.
- Step 3: Calculate base_fare = distance × 1.5.
- Step 4: If age < 12 or age > 60, base_fare = base_fare × 0.75.
- Step 5: Print final railway fare. Stop.

Flowchart



Source Code & Output

```
distance = int(input("Enter distance in km: "))
age = int(input("Enter passenger age: "))
base_fare = distance * 1.5
if age < 12 or age > 60:
    base_fare = base_fare * 0.75
print("Final Railway Fare: Rs.", base_fare)
```

```
Enter distance in km: 69 | Age: 19 | Final Railway Fare: Rs. 103.5
```

Case 4: Smart Parking Fee

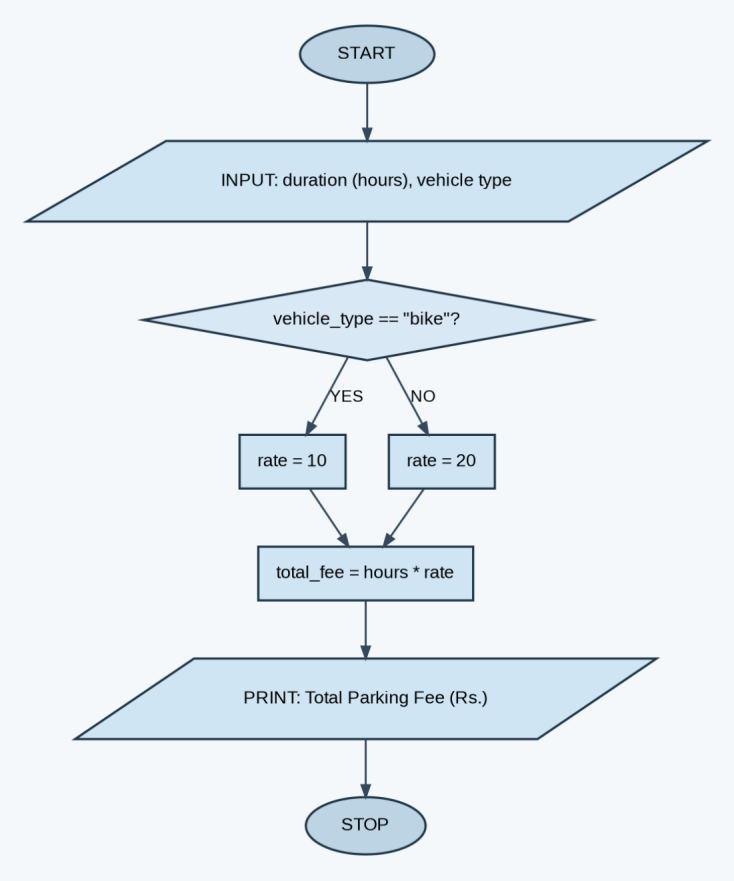
Problem Statement

Calculate total parking fees depending on vehicle type (Bike or Car) and duration in hours. The rate is Rs. 10 per hour for a bike and Rs. 20 per hour for any other vehicle.

Algorithm

- Step 1: Start.
- Step 2: Read parking duration (hours) and vehicle type from user.
- Step 3: Convert the vehicle type to lower case.
- Step 4: If vehicle type is "bike", rate = 10; otherwise rate = 20.
- Step 5: Calculate total_fee = hours × rate.
- Step 6: Print total parking fee. Stop.

Flowchart



Source Code & Output

```
hours = int(input("Enter duration in hours: "))
vehicle_type = input("Enter vehicle type (Bike/Car): ").lower()
rate = 10 if vehicle_type == "bike" else 20
total_fee = hours * rate
print("Total Parking Fee: Rs.", total_fee)
```

```
Duration: 3 | Type: bike | Total Parking Fee: Rs. 30
```

Case 5: Traffic Violation Penalty

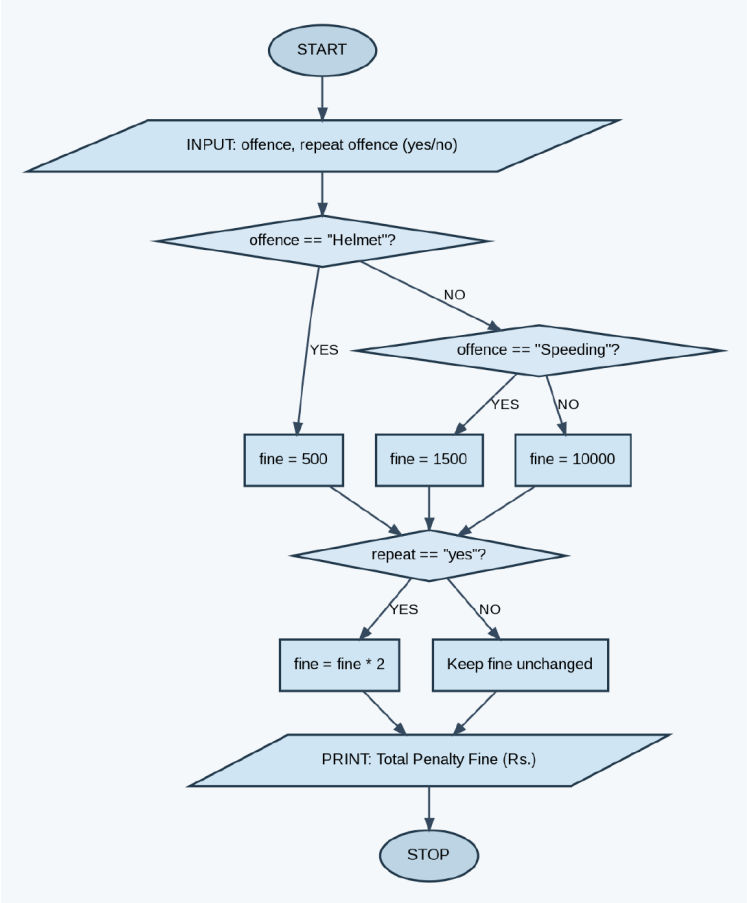
Problem Statement

Write a Python program to compute the penalty for a traffic violation. The base fine is Rs. 500 for a helmet offence, Rs. 1500 for speeding and Rs. 10000 for any other offence, and the fine is doubled for a repeat offence.

Algorithm

- Step 1: Start.
- Step 2: Read the offence and whether it is a repeat offence (yes/no).
- Step 3: If offence is "Helmet", fine = 500.
- Step 4: Else if offence is "Speeding", fine = 1500; otherwise fine = 10000.
- Step 5: If repeat is "yes", fine = fine × 2.
- Step 6: Print total penalty fine. Stop.

Flowchart



Source Code & Output

```
offence = input("Enter offence: ")
repeat = input("Is it a repeat offence? (yes/no): ")
fine = 500 if offence == "Helmet" else (1500 if offence == "Speeding" else 10000)
if repeat == "yes":
    fine *= 2
print("Total Penalty Fine: Rs.", fine)
```

```
Offence: Helmet | Repeat: no | Total Penalty Fine: Rs. 500
```

Case 6: Courier Charge Calculator

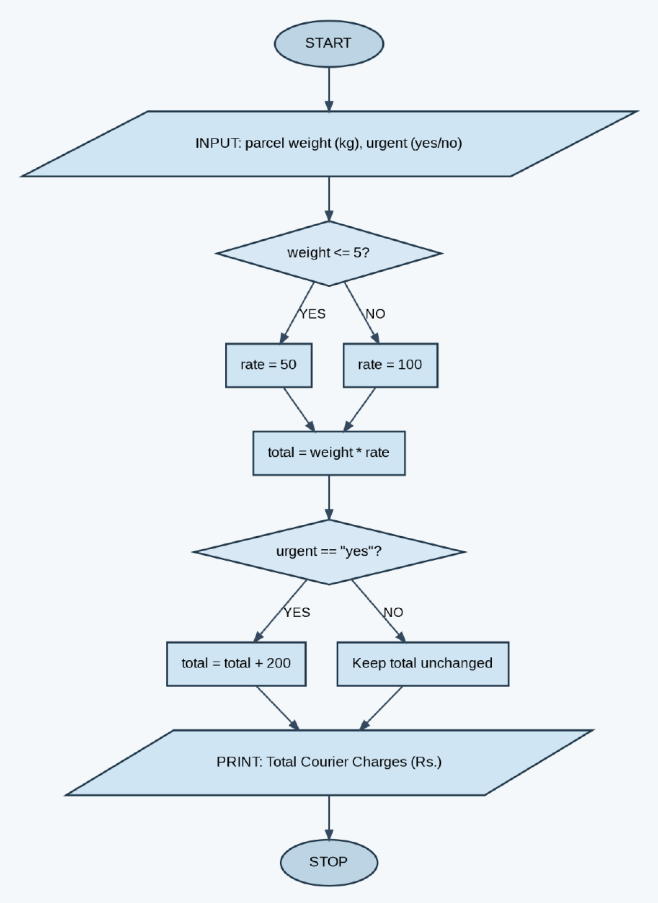
Problem Statement

Calculate courier charges based on parcel weight and urgency status. The rate is Rs. 50 per kg for parcels up to 5 kg and Rs. 100 per kg for heavier parcels, with a flat Rs. 200 added for urgent delivery.

Algorithm

- Step 1: Start.
- Step 2: Read parcel weight (kg) and urgency (yes/no) from user.
- Step 3: If weight ≤ 5, rate = 50; otherwise rate = 100.
- Step 4: Calculate total = weight × rate.
- Step 5: If urgent is "yes", total = total + 200.
- Step 6: Print total courier charges. Stop.

Flowchart



Source Code & Output

```
weight = float(input("Enter weight in kg: "))
urgent = input("Is it urgent? (yes/no): ")
rate = 50 if weight <= 5 else 100
total = weight * rate
if urgent == "yes":
    total += 200
print("Total Courier Charges: Rs.", total)
```

```
Weight: 30 | Urgent: yes | Total Courier Charges: Rs. 3200.0
```

Case 7: ATM Menu Simulation

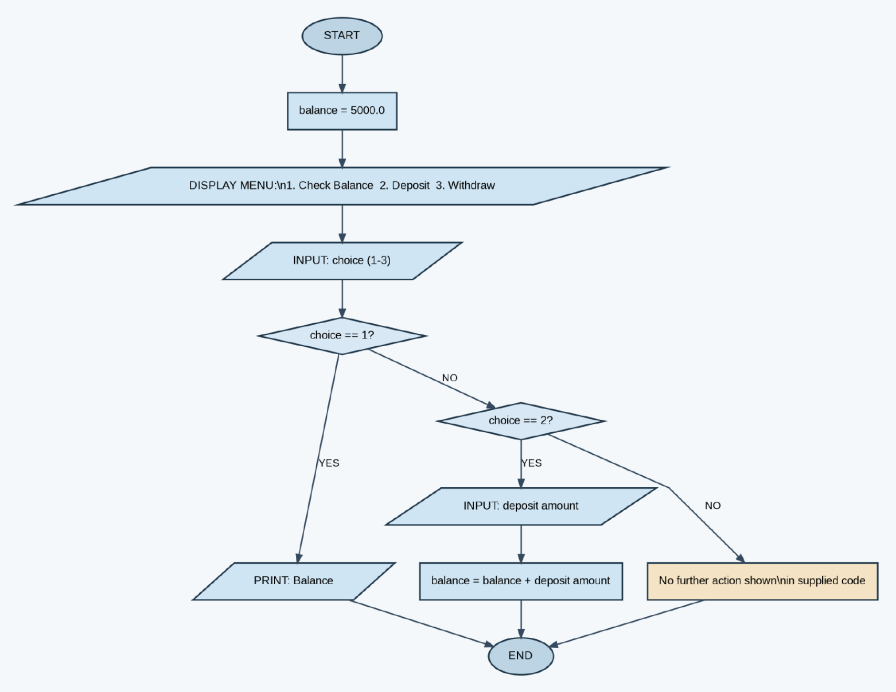
Problem Statement

Simulate a simple ATM menu with an opening balance of Rs. 5000. The user chooses to check the balance, deposit money, or withdraw; the supplied code implements the check-balance and deposit options.

Algorithm

- Step 1: Start.
- Step 2: Set balance = 5000.0.
- Step 3: Display the menu: 1. Check Balance, 2. Deposit, 3. Withdraw.
- Step 4: Read the user's choice (1-3).
- Step 5: If choice = 1, print the balance.
- Step 6: Else if choice = 2, read the deposit amount and add it to balance.
- Step 7: Otherwise no further action is shown in the supplied code. End.

Flowchart



Note: the supplied code shows only the check-balance and deposit branches; the withdraw branch is not present.

Source Code & Output

```
balance = 5000.0
print("1. Check Balance\n2. Deposit\n3. Withdraw")
choice = int(input("Enter choice (1-3): "))
if choice == 1:
    print("Balance:", balance)
elif choice == 2:
    balance += float(input("Deposit amount: "))
```

```
1. Check Balance | Choice: 1 | Balance: 5000.0
```

Case 8: Sports Tournament Points

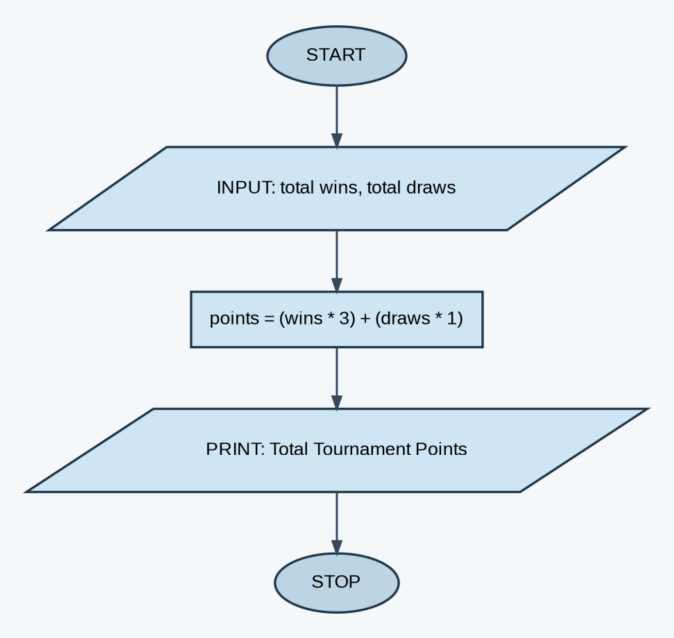
Problem Statement

Write a Python program to calculate a team's total tournament points from the number of matches won and drawn, awarding 3 points for each win and 1 point for each draw.

Algorithm

- Step 1: Start.
- Step 2: Read total matches won and total matches drawn.
- Step 3: Calculate points = (wins × 3) + (draws × 1).
- Step 4: Print total tournament points. Stop.

Flowchart



Source Code & Output

```
wins = int(input("Enter total matches won: "))
draws = int(input("Enter total matches drawn: "))
points = (wins * 3) + (draws * 1)
print("Total Tournament Points:", points)
```

```
Wins: 5 | Draws: 1 | Total Tournament Points: 16
```

Case 9: Transaction Audit using Tuples

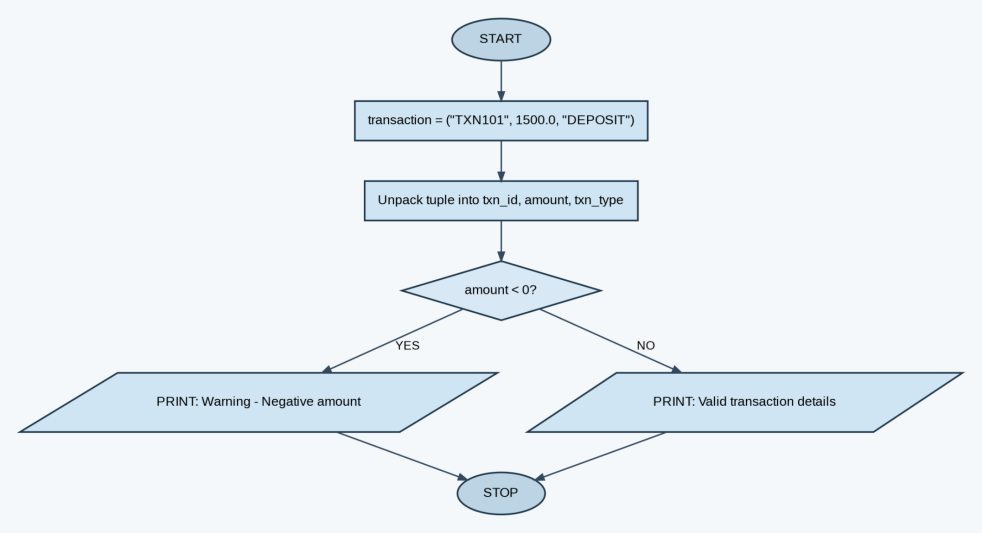
Problem Statement

Audit a financial transaction stored as a tuple (transaction ID, amount, type). Unpack the tuple and warn if the amount is negative; otherwise report the transaction as valid.

Algorithm

- Step 1: Start.
- Step 2: Create the tuple transaction = ("TXN101", 1500.0, "DEPOSIT").
- Step 3: Unpack the tuple into txn_id, amount and txn_type.
- Step 4: If amount < 0, print a warning for the negative amount.
- Step 5: Otherwise print the valid transaction details.
- Step 6: Stop.

Flowchart



Source Code & Output

```
transaction = ("TXN101", 1500.0, "DEPOSIT")
txn_id, amount, txn_type = transaction
if amount < 0:
    print("Warning: Negative amount")
else:
    print(f"Valid {txn_type} of Rs.{amount} recorded.")
```

```
Valid DEPOSIT of Rs.1500.0 recorded.
```

Case 10: Library Fine Calculator

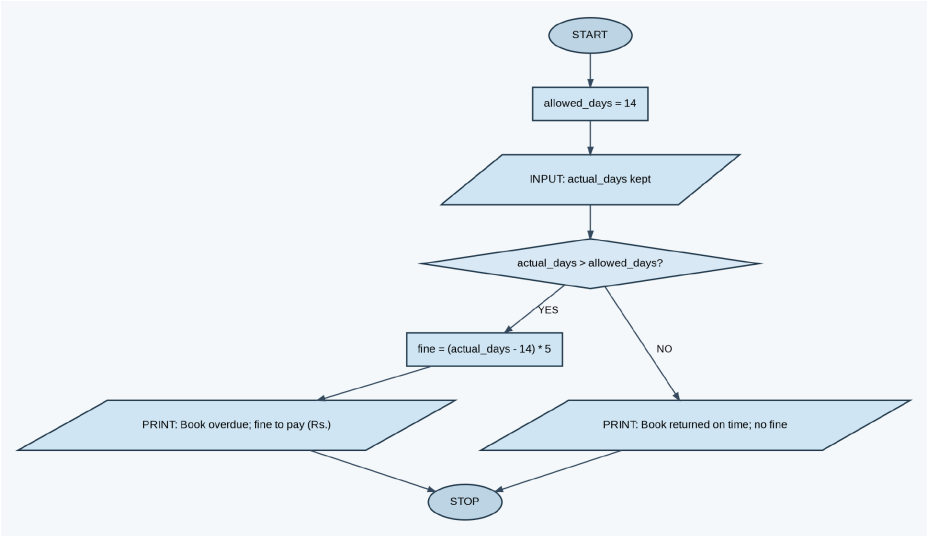
Problem Statement

Calculate the library fine for a book kept beyond the allowed period. A book may be kept for 14 days, and a fine of Rs. 5 per day is charged for every day beyond that.

Algorithm

- Step 1: Start.
- Step 2: Set allowed_days = 14.
- Step 3: Read the actual number of days the book was kept.
- Step 4: If actual_days > allowed_days, fine = (actual_days - 14) × 5 and print that the book is overdue with the fine.
- Step 5: Otherwise print that the book was returned on time with no fine.
- Step 6: Stop.

Flowchart



Source Code & Output

```
allowed_days = 14
actual_days = int(input("Enter total days kept: "))
if actual_days > allowed_days:
    fine = (actual_days - allowed_days) * 5
    print("Book is overdue. Fine to pay: Rs.", fine)
else:
    print("Book returned on time. No fine!")
```

```
Enter total days kept: 30 | Book is overdue. Fine to pay: Rs. 80
```